Islamic University of Technology

Board Bazar, Gazipur, Dhaka

SWE 4304 – Software Project Lab-I

Project Proposal

SMART GROCERY CLI

Grocery Shop Management System

SPL TEAM-10

SPL Team - 10

Team Members:

Ayman Rahman Bhuiyan
ID: 230042140
Bsc. in Software Engineering Department of Computer Science and Engineering

Arafat Hosen
ID: 230042151
Bsc. in Software Engineering Department of Computer Science and Engineering

Md. Jahirul Islam Shifat
ID: 230042156
Bsc. in Software Engineering Department of Computer Science and Engineering

Supervisor:

Mueeze Al Mushabbir
Lecturer
Department of Computer Science & Engineering
Islamic University of Technology

1. Project Overview & Motivation

1.1 Overview

Project Name: Smart Grocery CLI

Smart Grocery CLI is an offline, command-line based grocery management application designed to simplify inventory operations for small grocery shops. The system provides product management, user accounts, checkout functionality, recommendations, and analytics — all through a clean and structured CLI interface.

This project replaces traditional manual stock-keeping and paper-based purchase tracking with a digital file-based solution. Through a modular Java OOP architecture, the system ensures organized data management, easy product handling, secure authentication, and automated billing. Since the system is fully offline and uses file storage, it is suitable for small stores that do not rely on internet-based applications.

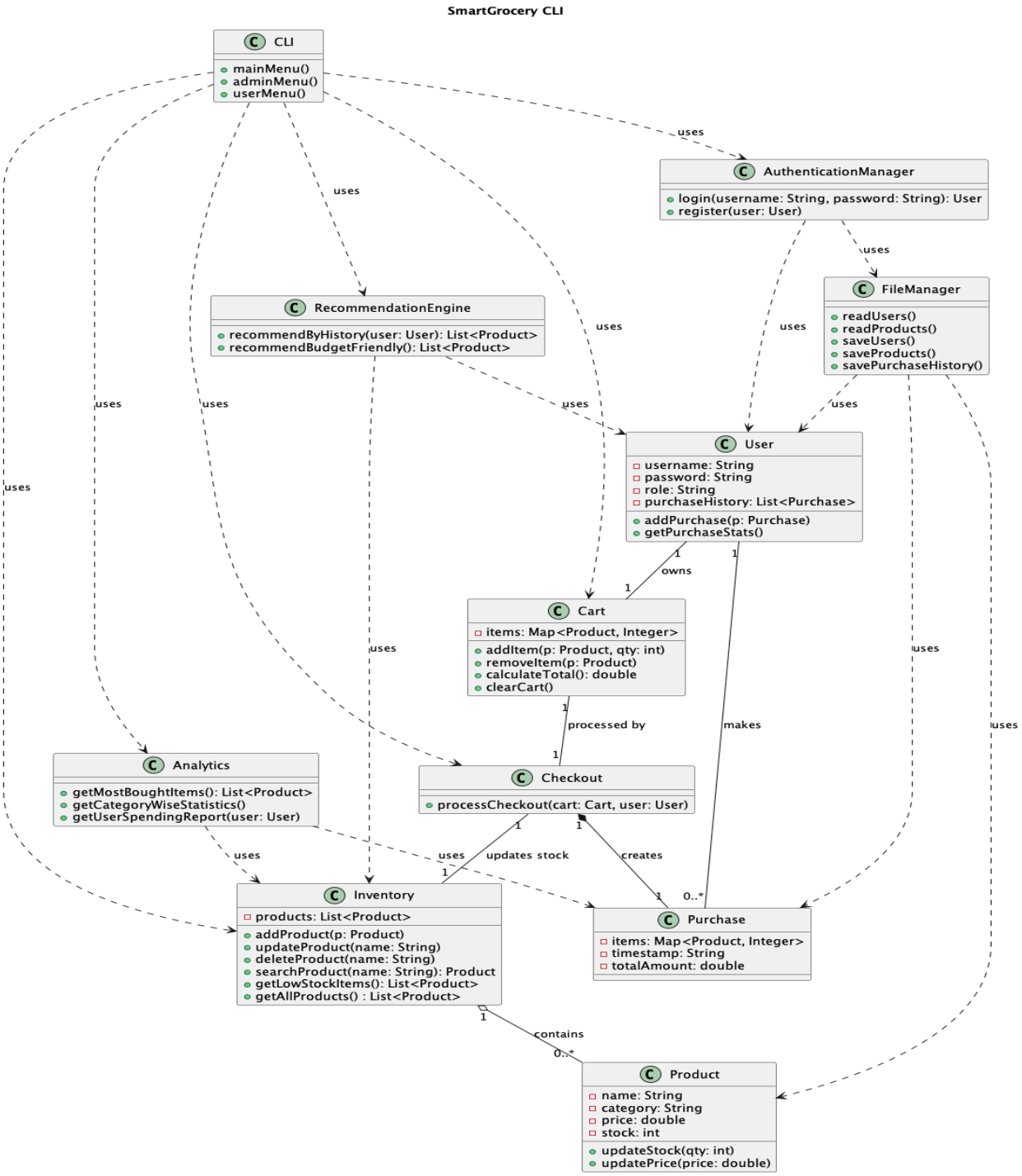
1.2 Motivation

Small grocery store owners in many local communities still depend heavily on manual stock management. This manual approach leads to frequent issues such as stock miscalculations, lost purchase records, and difficulty in tracking customer preferences.

Customers also face challenges since they cannot track their purchase history or get personalized recommendations. With no automated insights or analytics, store owners struggle to identify best-selling items, low-stock products, or user spending trends.

These real-world challenges inspired us to build Smart Grocery CLI—a simple but smart offline system that helps shop owners manage inventory effortlessly and helps customers receive better service without requiring internet connectivity.

2. UML DIAGRAM



UML Class Diagram Overview

The UML class diagram of **SmartGrocery CLI** illustrates the structural design of the system and the relationships among its core components. The system is designed following Object-Oriented principles to ensure modularity, low coupling, and clear responsibility separation.

The **Product** and **Inventory** classes handle grocery items and stock management, where a single inventory contains multiple products. The **User** class represents system users (Admin and Customer) and maintains purchase history, while each user owns a **Cart** to manage selected items before checkout.

The **Checkout** class processes purchases by validating cart items, updating inventory stock, and creating **Purchase** records. Each completed checkout results in one purchase entry, which is stored for future reference and analysis.

User authentication and registration are handled by the **AuthenticationManager**, separating security logic from the user interface. Persistent data storage is managed through the **FileManager**, which reads and writes user, product, and purchase information using local text files.

Additional functionality is provided by the **RecommendationEngine** and **Analytics** classes, which generate offline product recommendations and system reports based on purchase and inventory data.

The **CLI** acts as the presentation layer, providing role-based menus and handling user interaction. It depends on core system classes to execute operations but remains independent of business logic, ensuring a clean layered architecture.

Note on Primary and Foreign Keys

Primary keys and foreign keys are not explicitly identified in the UML class diagram because the SmartGrocery CLI system does not use a relational database. Instead, the system operates using object-oriented in-memory objects with file-based storage. Relationships between entities are maintained through object references rather than database keys.

3. Key Features

The **SmartGrocery CLI** system integrates offline inventory management, user-based purchasing, and intelligent analytics to support efficient grocery management without relying on internet connectivity or external databases. The system is designed as a headless, command-line-based application that emphasizes strong Object-Oriented Design, modularity, and data integrity through file-based storage.

The following sections describe the major functionalities implemented in the system.

3.1 User Management

The user authentication module ensures secure access to the system by verifying user credentials and managing role-based permissions. Authentication is handled through a username–password mechanism, and all user data is stored locally in text files, ensuring complete offline functionality.

● Username and Password Authentication

- Users can register and log in using valid credentials.
- Authentication verifies user identity before granting access to the system.
- User information is stored securely in local files.

● Role-Based Access Control

- The system supports two user roles:
    - **Admin**: Responsible for managing inventory and system data.
    - **Customer**: Allowed to browse products, make purchases, and view personal history.
- Each role is provided access only to permitted functionalities.

3.2 Product & Inventory Management System

This module is the core of the SmartGrocery CLI system and allows administrators to manage grocery products and stock levels efficiently. All inventory data is maintained offline using file-based storage.

● Inventory Management Features

- Add new grocery products with name, category, price, and stock quantity.
- Update existing product details such as price and available stock.
- Delete products that are discontinued or incorrectly added.
- Categorize items into groups such as fruits, vegetables, dairy, and snacks.
- View the complete inventory in a structured CLI table format.

● Low-Stock Alert System

- The system automatically identifies products with stock below a predefined threshold.
- Low-stock warnings are displayed to alert administrators about refill requirements.

3.3 User Purchase & Cart Management

This module allows customers to manage their shopping activities efficiently. Each user is associated with a personal cart and purchase history.

● Cart Management Features

- Add products to the cart with selected quantities.
- Remove items or modify quantities in the cart.
- View cart contents and calculate the total cost dynamically.

● Checkout Process

- Generate a bill during checkout.
- Validate product availability before finalizing purchases.
- Automatically update inventory stock after a successful purchase.
- Save purchase details to the user's purchase history file.

3.4 Offline Recommendation Engine

The recommendation engine provides intelligent product suggestions based on user purchase behavior. All recommendations are generated offline using locally stored data.

● Recommendation Features

- Suggest products based on previous purchase history.
- Recommend items from frequently purchased categories.
- Offer budget-friendly alternatives within the same product category.

This feature enhances user experience without requiring internet access or external APIs.

3.5 Analytics & Reporting System

The analytics module provides meaningful insights into system usage and purchasing patterns. These reports assist both administrators and users in understanding trends and making informed decisions.

● System-Wide Analytics

- Identify the most frequently purchased products.
- Generate category-wise purchase statistics.
- Display low-stock product reports.

● User-Specific Reports

- Show total spending per user.

- Display recent purchase summaries.
- Provide simple weekly or monthly spending reports.

3.6 File-Based Data Management System

SmartGrocery CLI uses a file-based storage system to ensure data persistence and offline accessibility. All system data is stored in structured text files.

● Data Storage Features

- Store user information in users.txt.
- Maintain product and inventory data in products.txt.
- Record purchase history in purchases.txt.
- Save cart and transactional data as required.

● Data Integrity

- Files are read and written using controlled access methods.
- Data is updated consistently after each operation to avoid inconsistencies.

3.7 Command-Line Interface (CLI) Interaction

The system operates entirely through a terminal-based interface, complying with SPL-1 headless application requirements.

● CLI Capabilities

- Display structured menus for admin and customer roles.
- Provide guided input prompts for user actions.
- Show system messages, alerts, and reports clearly.

4. Tools & Technologies

4.1 Programming Language

We used Java as the primary programming language because it supports strong OOP architecture, cross-platform CLI development, and built-in file handling mechanisms essential for offline storage.

4.2 Development Environment

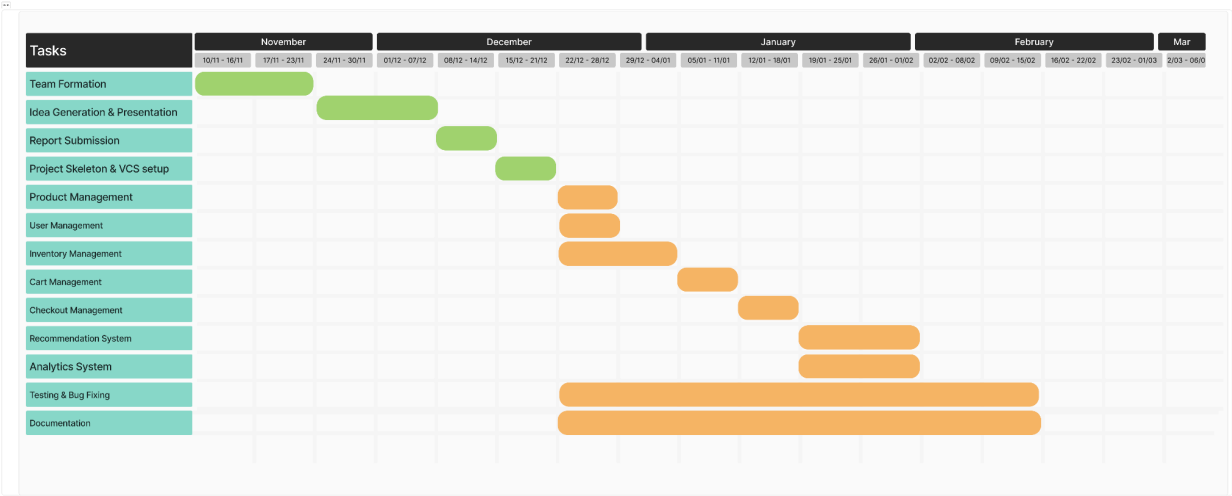
- VS Code — used for writing and running Java code with extensions for debugging and formatting
- Java Compiler (JDK) — to compile and execute code

4.3 Version Control

- Git for version tracking
- GitHub as the central remote repository to collaborate and maintain project updates
- This version control workflow ensures smooth teamwork, code reviews, and revision management.

5. Proposed Project Timeline

Timeline Diagram :



Planning and Preparation (Week 1-6)

During the first six weeks of the project, our focus was on building a strong foundation. The team was formed and roles were discussed to ensure smooth collaboration. We generated project ideas, analyzed real-world problems, and finalized the project concept through discussions and feedback. After idea finalization, we prepared and delivered the project proposal presentation. Finally, we completed the initial report submission, designed the basic project skeleton, and set up version control using Git and GitHub to manage code and track progress effectively.

Product Management Module (Week 6–7)

In this phase, the product-related functionalities are implemented. The Product and Inventory classes are developed to handle grocery item information such as name, category, price, and stock quantity. Features for adding, updating, deleting, and viewing products are implemented. File-based

storage mechanisms are integrated to persist product data. Initial validation and error-handling logic are also introduced to ensure data consistency.

User Management Module (Week 7–8)

This phase focuses on implementing user-related functionalities. The User and AuthenticationManager classes are developed to support user registration and login. Role-based access control is introduced to differentiate between Admin and Customer users. Purchase history tracking is initialized for each user. User data is stored and retrieved using file-based storage, and authentication workflows are tested through the CLI interface.

Inventory Management Module (Week 6–8)

During this phase, inventory control logic is enhanced. Stock tracking and low-stock alert mechanisms are implemented to notify administrators when product quantities fall below a defined threshold. Inventory update operations are integrated with product management and checkout processes. This phase ensures that inventory data remains accurate and synchronized after each purchase.

Cart & Checkout System (Week 8–10)

This phase implements the core purchasing workflow. The Cart class is developed to manage selected products and quantities for each user. The Checkout class is implemented to validate product availability, calculate total costs, update inventory stock, and generate purchase records. Purchase data is saved in files, and user purchase histories are updated accordingly. The complete purchase flow is tested to ensure reliability and correctness.

Recommendation & Analytics Module (Week 11–13)

In this phase, intelligent features are added to enhance user experience. The RecommendationEngine analyzes user purchase history to suggest relevant products and budget-friendly alternatives. The Analytics module is implemented to generate reports such as most bought items, category-wise statistics, user spending summaries, and low-stock reports. All computations are performed offline using locally stored data.

Documentation, Testing & Bug Fixing (Week 6–14)

This phase runs continuously throughout the project timeline. Tests are performed after each module implementation to verify functionality and catch defects early. Bugs and performance issues are identified and resolved iteratively. Project documentation, including UML diagrams, feature descriptions, and user guides, is prepared and refined. Version control practices are maintained using Git to track progress and individual contributions.

6. Action taken based on the feedback

Based on the feedback provided by the judges, we addressed the following points:

1. Remove supervisor information from the presentation slides
2. Add page number on slide
3. **Focus on Recommendation System**
    - During the proposal presentation, the teachers pointed out that although a recommendation feature was included in the project, the **algorithm or process behind the recommendation system was not clearly defined**. The judges suggested using a more efficient and structured recommendation approach to improve clarity and effectiveness.
    - **Action Taken:**
      Based on this feedback, we refined the recommendation logic to use **simple, understandable, and efficient rule-based techniques** that are suitable for our current academic level. The recommendation process is now clearly defined using purchase frequency analysis, category-based matching, and price comparison within the same category. Advanced algorithms were intentionally avoided, as they are beyond the scope of the current course. The updated approach has been clearly documented to ensure transparency and clarity.

4. **Timeline Incorrect**
    - Initially, the project timeline incorrectly showed all development phases starting from the first week, including implementation, testing, and documentation. This did not reflect a realistic or structured development process.

- **Action Taken:**
  The project timeline has been corrected with proper week-wise distribution of phases. Feature implementation, testing, and documentation are now aligned logically and progressively across the project duration. Each development phase now starts at an appropriate time, ensuring a more accurate and realistic project workflow.